

Laboratorio de Computación

Salas A y B

Profesor(a): Carlos Chaves Mercado

Asignatura: Fundamentos de programación

Grupo: 8

No de Práctica(s): Práctica de estudio 12: Lecturas y escritura de atos

Integrante(s): Cruz Gómez Lizette Jacqueline

No. de lista o brigada: Mesa 6

Semestre: 2026-2

Fecha de entrega: 15 de mayo del 2026

Observaciones:

CALIFICACIÓN: _____

Objetivo:

El alumnado elaborará programas en C donde la solución del problema se divida en funciones. Distinguirá lo que es el prototipo o firma de una función y la implementación de ella, asimismo manipulará parámetros tanto en la función principal como en otras.

Introducción:

El paradigma de la programación estructurada fundamenta su eficiencia en la modularidad, un principio de diseño de software que permite fragmentar un problema complejo en subproblemas más pequeños, independientes y manejables. En el lenguaje C, este esquema se implementa operativamente mediante el uso de funciones. Una función se define como un bloque de código autónomo y reutilizable, diseñado para ejecutar una tarea específica y bien delimitada, el cual puede ser invocado desde la función principal `main` o desde otros módulos del sistema.

La integración formal de una función dentro de un programa estructurado exige el dominio de tres componentes secuenciales: el prototipo o declaración, que notifica al compilador la firma del módulo antes de su uso; la definición, que contiene la codificación del algoritmo y las variables locales asociadas; y la llamada o invocación, que transfiere temporalmente el hilo de ejecución del procesador hacia el bloque modular. La comunicación de datos entre estas entidades se administra estrictamente a través de los parámetros, distinguiendo entre el paso por valor —donde se transmite una copia del dato protegiendo la variable de origen— y el paso por referencia, el cual explota el uso de apuntadores para interactuar de forma directa con las direcciones de memoria física. La correcta modularización de los algoritmos no solo optimiza la legibilidad del código fuente, sino que disminuye significativamente la redundancia, facilita el aislamiento de errores durante la depuración y eleva el índice de reutilización de los componentes lógicos en la ingeniería de software.

Desarrollo:

Programa 1.c

- Muestra la estructura básica de una función sin retorno (`void`). Enseña cómo declarar un prototipo al inicio del archivo para que la función principal conozca su existencia, y cómo llamarla para ejecutar un bloque de código aislado.

```
8
9  #include <stdio.h>
10 #include <string.h>
11
12 void imprimir(char[]);
13
14 int main(){
15     char nombre[] = "Facultad de Ingeniería";
16     imprimir(nombre);
17 }
18
19 void imprimir(char s[]){
20     int tam;
21     for ( tam=strlen(s)-1 ; tam>=0 ; tam-- )
22         printf("%c", s[tam]);
23     printf("\n");
24 }
```



Figura 1.1: Flujo de control y enlace de prototipos en la arquitectura modular (Programa 1c)

Programa 2.c

- Ilustra el mecanismo de **paso por valor**. Demuestra que cuando envías variables como argumentos a una función, esta recibe solo una copia, por lo que cualquier modificación interna no afecta en absoluto a las variables originales del main.

```
8
9 #include <stdio.h>
10
11 void sumar();
12 int main()
13 {
14     sumar();
15 }
16 void sumar()
17 {
18     int z, x=5, y=10;
19     z=x*y;
20     printf("%i",z);
21 }
```

15

...Program finished with exit code 0
Press ENTER to exit console.

Figura 1.2: Mecanismo de aislamiento de datos en el paso por el valor (Programa 2c)

Programa 3.c

- Enseña el uso de la instrucción return. Muestra cómo diseñar funciones que procesan datos internamente y devuelven un único resultado numérico directo a la función que las invocó, permitiendo usar la llamada dentro de operaciones aritméticas.

```
8
9 #include <stdio.h>
10
11 int resultado;
12
13 void multiplicar();
14
15 int main()
16 {
17     multiplicar();
18     printf("%i",resultado);
19     return 0;
20 }
21 void multiplicar()
22 {
23     resultado = 5 * 4;
24 }
```

20

...Program finished with exit code 0
Press ENTER to exit console.

Figura 1.3: Retorno de magnitudes hacia el entorno llamante (Programa 3c)

Programa 4.c

- Ejemplifica el concepto de **variables locales** y su ciclo de vida. Demuestra que las variables creadas dentro de una función solo existen mientras esa función se esté ejecutando en la pila de memoria; una vez que termina, se destruyen y son inaccesibles desde fuera.

```

8
9 #include<stdio.h>
10
11 void incremento();
12
13 int enteraGlobal;
14
15 int main()
16 {
17     int cont;
18     enteraGlobal = 0;
19     for (cont=0 ; cont<5 ; cont++)
20     {
21         incremento();
22     }
23     return 0;
24 }
25 void incremento()
26 {
27     int enteraLocal = 5;
28     enteraGlobal += 2;
29     printf("global(%i) + local(%i) = %d\n",enteraGlobal, enteraLocal, enteraGlobal+enteraLocal);
30 }

```

global(2) + local(5) = 7
global(4) + local(5) = 9
global(6) + local(5) = 11
global(8) + local(5) = 13
global(10) + local(5) = 15

...Program finished with exit code 0
Press ENTER to exit console.

Figura 1.4: Ciclo de vida y destrucción de marcos de activación locales (Programa 4c)

Programa 5.c

- Expone el funcionamiento de las **variables globales**. Muestra que al declararlas fuera de cualquier función, se almacenan en un espacio permanente de memoria y pueden ser leídas o modificadas por cualquier módulo del programa sin necesidad de usar parámetros.

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23

```

Online C Compiler.
Code, Compile, Run and Debug C program online.
Write your code in this editor and press "Run" button to compile and execute it.

```

9 #include<stdio.h>
10 #include <string.h>
11 int main(int argc, char** argv)
12 {
13     if (argc == 1)
14     {
15         printf("El programa no contiene argumentos.\n");
16         return 0;
17     }
18     printf("Los elementos del arreglo argv son:\n");
19     for (int cont = 0 ; cont < argc ; cont++){
20         printf("argv[%d] = %s\n", cont, argv[cont]);
21     }
22     return 0;
23 }

```

El programa no contiene argumentos.

...Program finished with exit code 0
Press ENTER to exit console.

Figura 1.5: Persistencia y accesibilidad del segmento de datos globales (Programa 5c)

Programa 6.c

- Introduce el **paso por referencia** mediante apuntadores. Muestra cómo enviar la dirección de memoria de una variable usando el operador **&** para que la función, a través de variables tipo puntero *****, pueda modificar directamente el valor original en el **main**.

```

6
7 //*****
8
9 #include<stdio.h>
10 void llamarFuncion();
11
12 int main()
13 {
14     for (int j=0 ; j < 5 ; j++)
15     {
16         llamarFuncion();
17     }
18 }
19
20 void llamarFuncion()
21 {
22     static int numVeces = 0;
23     printf("Esta función se ha llamado %d veces.\n",++numVeces);
24 }

```

Input

```

Esta función se ha llamado 1 veces.
Esta función se ha llamado 2 veces.
Esta función se ha llamado 3 veces.
Esta función se ha llamado 4 veces.
Esta función se ha llamado 5 veces.

...Program finished with exit code 0
Press ENTER to exit console.

```

Figura 1.6: Modificación directa mediante el paso por referencia y apuntadores ((Programa 6c)

funcEstatica.c

- Demuestra el uso del modificador static en variables locales. Explica que al volver una variable estática, esta no se destruye al salir de la función, sino que retiene su último valor en la memoria fija para la siguiente vez que la función sea invocada.

```

main.c
11 int suma(int,int);
12 static int resta(int,int);
13
14 int producto(int,int);
15 static int cociente (int,int);
16
17 int suma (int a, int b)
18 {
19     return a + b;
20 }
21 static int resta (int a, int b)
22 {
23     return a - b;
24 }
25 int producto (int a, int b)
26 {
27     return (int)(a*b);
28 }
29 static int cociente (int a, int b)
30 {
31     return (int)(a/b);
32 }
33 int main() {
34     int x = 10, y = 15;
35     printf("Cociente:%d\n", cociente (x,y));
36     return 0;
37 }
38
39
40

```

Cociente:0

```

...Program finished with exit code 0
Press ENTER to exit console.

```

Figura 1.7: Retención de estado y permanencia en memoria estática (funcestatica)

calculadora.c

- Integra todos los conceptos en una aplicación práctica. Utiliza un menú iterativo para coordinar múltiples funciones independientes (suma, resta, multiplicación, división), demostrando cómo la modularidad permite construir un código limpio, organizado y fácil de mantener.

```

8  //##### calculadora.c #####
9  #include <stdio.h>
10
11  int suma(int,int);
12  int producto(int,int);
13
14  int main()
15  {
16      printf("5 + 7 = %i\n",suma(5,7));
17      printf("6 * 8 = %i\n",producto(6,8));
18
19      return 0;
20  }
21
22  int suma (int a, int b)
23  {
24      return a + b;
25  }
26
27  int producto(int a, int b)
28  {
29      return a * b;
30  }

```

5 + 7 = 12
6 * 8 = 48

...Program finished with exit code 0
Press ENTER to exit console.

Figura 1.8: Arquitectura jerárquica de software y coordinación de menús (calculadora.c)

Análisis de resultados:

Programa 1.c

El análisis de este programa se orienta a la implementación de la declaración formal de funciones mediante prototipos y su posterior invocación desde el flujo principal. Al colocar el prototipo en la sección global del archivo, se notifica explícitamente al compilador la firma de la función, detallando el tipo de retorno y la lista de parámetros requeridos. Esto permite que la función principal pueda invocar el módulo de forma segura antes de que el compilador lea el cuerpo completo de su definición. Durante la ejecución, la llamada transfiere el control del programa hacia la subrutina, activando su propio marco en la pila de memoria, y retorna de forma secuencial al terminar la tarea, demostrando la estructura básica de la modularidad sin retorno de valores.

Programa 2.c

Este módulo examina el flujo de datos y la transferencia de parámetros formales por valor hacia una función encargada de realizar un procesamiento específico. Cuando el programa principal invoca a la función pasando variables como argumentos, el sistema operativo realiza una copia exacta de sus magnitudes y las aloja en las direcciones de memoria temporal asignadas a los parámetros del módulo receptor. Debido a esta duplicación en el hardware, cualquier modificación aritmética o asignación realizada dentro de la función afecta exclusivamente a la variable local de la subrutina; las variables originales del entorno llamante permanecen intactas y protegidas contra alteraciones destructivas colaterales.

Programa 3.c

El análisis técnico de este código se centra en la anatomía de las funciones que calculan y devuelven un único valor a través de la instrucción de retorno `return`. A diferencia de los procedimientos declarados como vacíos, estas funciones actúan como expresiones evaluables dentro del programa; la Unidad Aritmético Lógica de la CPU procesa las operaciones internas del

módulo y, al encontrar la sentencia de escape, inyecta directamente la magnitud resultante en la línea de código donde se originó la llamada. Este diseño permite capturar el resultado computado para asignarlo a una variable del entorno principal o integrarlo directamente en operaciones lógicas complejas de manera limpia.

Programa 4.c

Este ejercicio analiza el comportamiento de las variables de ámbito o alcance local y el ciclo de vida de los datos dentro del entorno de ejecución. Las variables declaradas en el cuerpo de una función nacen de forma dinámica en la pila de memoria únicamente en el instante en que el módulo es invocado por el hilo del programa; una vez que la función concluye su bloque de instrucciones y devuelve el control al código llamante, su marco de activación se destruye por completo y la memoria es liberada. El programa demuestra que estas variables son completamente invisibles e inaccesibles fuera de su propio bloque, impidiendo que otros componentes del software alteren accidentalmente sus valores.

Programa 5.c

El análisis de este programa aborda la naturaleza y los riesgos de la manipulación de variables globales dentro del desarrollo de software estructurado. Al definir una variable fuera del alcance de cualquier función, el compilador le asigna una dirección permanente en el segmento de datos de la memoria física, otorgándole un tiempo de vida que persiste durante toda la ejecución del programa. Esto permite que cualquier función del código pueda leer o modificar su valor de forma directa sin necesidad de pasar parámetros; sin embargo, el programa evidencia cómo este acceso indiscriminado debilita el encapsulamiento y eleva la probabilidad de errores lógicos difíciles de depurar debido al acoplamiento de los módulos.

Programa 6.c

Este fragmento técnico analiza el paso de parámetros por referencia explícita mediante el uso de apuntadores en la firma de la función. En lugar de duplicar el valor numérico de la variable, el programa principal transmite la dirección de memoria física del dato utilizando el operador de referencia. La función receptora se declara utilizando variables de tipo puntero para almacenar estas direcciones; al aplicar el operador de indirección dentro del algoritmo modular, la CPU accede directamente a las celdas de memoria originales de la función principal, permitiendo que las modificaciones se apliquen de forma directa en el origen y logrando que un módulo altere múltiples variables simultáneamente.

Funcestatica

El análisis de este código evalúa el impacto de la palabra reservada `static` aplicada a variables locales dentro de un entorno funcional. A diferencia de una variable local convencional que se destruye al salir de la subrutina, el modificador `static` altera la política de gestión de memoria del compilador, instruyendo al sistema operativo para que retenga la dirección y el valor de la variable en el segmento de datos fijos durante toda la vida del proceso. Cada vez que la función vuelve a ser invocada, la variable no se reinicializa, sino que conserva intacto el último estado alcanzado en la iteración previa, permitiendo implementar contadores internos o registros de estado persistentes sin necesidad de recurrir a variables globales.

Calculadora.c

El análisis de este programa final examina la integración de múltiples funciones modulares coordinadas mediante una estructura de control de selección múltiple tipo menú. El diseño de la aplicación explota la modularidad para aislar cada operación aritmética básica (suma, resta, multiplicación y división) en funciones independientes con firmas específicas de entrada y salida de datos. El cuerpo principal del programa implementa un ciclo iterativo controlado que despliega las opciones y captura la selección del usuario; mediante una sentencia condicional o de bifurcación, se invoca de manera selectiva al módulo correspondiente pasando los operandos requeridos, demostrando cómo la abstracción funcional permite estructurar aplicaciones extensas, escalables, limpias y fáciles de mantener.

Conclusión:

Cruz Gomez Lizette Jacqueline: La Práctica 11 demuestra que la modularidad mediante el uso de funciones es una estrategia fundamental en la ingeniería de software para desarrollar código estructurado, legible y escalable. A través de la experimentación con los diversos programas, se comprobó que el control del alcance de las variables (locales, globales y estáticas) y la correcta selección del mecanismo de transmisión de parámetros (por valor para proteger la integridad de los datos, o por referencia para la modificación directa en memoria mediante apuntadores) determinan la eficiencia y la seguridad de un algoritmo. En última instancia, la centralización de operaciones específicas en módulos independientes, como se evidenció en el diseño de la calculadora, simplifica la depuración de errores y maximiza la reutilización de componentes lógicos en la construcción de sistemas complejos.

Bibliografía

El lenguaje de programación C. Brian W. Kernighan, Dennis M. Ritchie, segunda edición, USA, Pearson Educación 1991.